Verdict: What We've Built So Far

Plain explanation + technical detail for Part 1 and Part 2 — September 2026

Verdict is a feature for Gridline. The big idea: F1 stewards hand out penalties after races, and they publish a PDF document explaining each one. This project reads those documents automatically, saves the important details, and — eventually — will compare them against real race data to see if similar incidents get treated the same way. So far, we've finished the first two of seven planned parts. Each part below has a plain-language explanation first, followed by the actual code and technical detail behind it.

Part 1: Teaching the Computer to Read the Documents

A PDF is just a document file — like the ones FIA (the sport's governing body) publishes after every penalty decision. But a PDF is just text sitting on a page; the computer doesn't automatically know which part is the driver's name, which part is the penalty, and which part is just the reasoning. So we built a program that opens each PDF, reads through it line by line, and sorts the text into the right buckets — almost like highlighting a form with different colored highlighters for each section.

It figures out things like: which driver and car were involved, what corner it happened in, what the other car's number was, what the penalty was (if any), and how many penalty points were added. We tested it against four real FIA documents that were all worded differently, and fixed several tricky mistakes along the way.

- It correctly tells apart a real penalty, a warning, and “no further action” — three very different outcomes.
- It doesn't get confused when FIA uses an old word (“Offence”) instead of a newer one (“Infringement”) for the same thing — it treats them as the same field.
- If the document doesn't clearly name the other car in the main summary, it knows to check the longer explanation paragraph instead.
- We also wrote automatic checks (called tests) that re-verify all four documents parse correctly every time we change the program — so we don't have to manually re-check by hand each time.

Technical Detail: How the Parser Actually Works

File: `verdict/parser/parse_decision.py`. The FIA documents always use the same fixed labels (“Fact”, “Decision”, “Reason”, etc.), always in the same order, so the parser doesn't need real AI/NLP — it just needs to recognize where one labeled section ends and the next begins, then extract a few specific values with `regex` (a pattern-matching syntax for finding text shapes, like “a number right after the word Turn”).

1. Normalizing label wording across seasons. Older documents say “Offence”, newer ones say “Infringement” for the same field — this dictionary maps both to one consistent key so downstream code never has to care which word a given document used:

```
LABEL_MAP = {
    "No / Driver": "Driver",
    "Competitor": "Competitor",
    "Time": "Time",
    "Session": "Session",
    "Fact": "Fact",
    "Infringement": "Infringement",
    "Offence": "Infringement",   # older FIA wording, same meaning
    "Decision": "Decision",
    "Reason": "Reason",
}
```

2. The core parsing loop. It walks through the document line by line. Whenever a line starts with a known label, it closes out whatever section was open and starts a new one; otherwise the line is added to whichever section is currently open. The boundary check (`not line[len(label)].islower()`) matters because "Decision" is also the start of the word "Decisions" (part of FIA's standard footer text) — without it, that footer would be wrongly detected as a new "Decision" field and silently overwrite the real one:

```
def parse_decision(pdf_path):
    lines = get_lines(pdf_path)
    result = {}
    current_label = None
    buffer = []

    for line in lines:
        matched_label = None
        for label in LABEL_MAP:
            if line.startswith(label) and (
                len(line) == len(label) or not line[len(label)].islower()
            ):
                matched_label = label
                break

        if matched_label:
            if current_label:
                result[current_label] = " ".join(buffer).strip()
            current_label = LABEL_MAP[matched_label]
            buffer = [line[len(matched_label):].strip()]
        elif current_label:
            buffer.append(line.strip())

    if current_label:
        result[current_label] = " ".join(buffer).strip()

    result["Reason"] = result["Reason"].split("Competitors are reminded")[0].strip()
    return result
```

3. Pulling structured numbers out of free text. Once the raw fields are separated, small regex helpers extract the actual data points — turn number, the driver's own car, the other car involved (falling back to the Reason paragraph if Fact doesn't mention a second car), and penalty points:

```
def extract_turn(fact_text):
    match = re.search(r"[Tt]urn\s+(\d+)", fact_text)
    return int(match.group(1)) if match else None

def extract_other_car(fact_text, reason_text, own_car):
    for text in (fact_text, reason_text):
        cars = [int(n) for n in re.findall(r"[Cc]ar\s+(\d+)", text)]
        others = [c for c in cars if c != own_car]
        if others:
            return others[0]
    return None
```

Automated tests (file: `verdict/parser/test_parse_decision.py`) lock in the correct answer for each of the four real documents tested, so any future change to the parser gets checked in one command instead of by

hand:

```
def test_grid_drop_penalty():
    result = parse_decision("verdict/data/raw/test.pdf")
    assert result["own_car"] == 77
    assert result["other_car"] == 20
    assert result["penalty_points"] == 3
    assert result["turn"] == 6
```

Part 2: Saving the Information Properly

Once the computer pulls the details out of a PDF, that information needs somewhere to live permanently — otherwise it disappears the moment the program finishes running. We did this in two steps.

Step one: save each decision as a small file called JSON — think of it as a simple, readable text file that stores information as labeled pairs, like a digital index card (“Driver: Lewis Hamilton”, “Turn: 6”, and so on). This was a good first step because it's easy to open and check by eye, without committing to anything permanent yet.

Step two: once we were confident the information looked right, we built a proper database table — a structured, permanent storage system (the same kind of system banks, apps, and websites use to store their data) — and wrote a small program that reads every saved JSON file and loads it into that database.

- Every parsed penalty decision now has a permanent home in a real database, not just scattered files.
- The loading program is safe to run more than once — it won't accidentally create duplicate copies of the same decision.
- Along the way we found and fixed a real bug: on Windows, folder names were sneaking into the saved data by mistake, and we corrected it.

Technical Detail: JSON, the Database Table, and the Loader

1. Saving as JSON. File: `verdict/parser/parse_decision.py`. Python's built-in `json` module converts the parsed dictionary straight into a text file:

```
def save_as_json(data, output_path):
    with open(output_path, "w") as f:
        json.dump(data, f, indent=2)

def get_output_path(pdf_path):
    filename = os.path.basename(pdf_path)
    json_filename = filename.replace(".pdf", ".json")
    return os.path.join("verdict/data/parsed", json_filename)
```

2. The database table. File: `db/sql/010_penalty_decisions.sql`. Follows the same migration pattern already used elsewhere in Gridline's database (`db/sql/001` through `009`). The unique index on `source_file` is what makes it safe to reload the same PDF twice without creating a duplicate row:

```
CREATE TABLE penalty_decisions (
  decision_id BIGSERIAL PRIMARY KEY,
  source_file TEXT NOT NULL,
  driver_text TEXT,
  own_car INT,
  other_car INT,
```

```
  competitor TEXT,
  session TEXT,
  incident_time TEXT,
  turn INT,
  fact TEXT,
  infringement TEXT,
  decision TEXT,
  penalty_points INT,
  reason TEXT
);

CREATE UNIQUE INDEX uq_penalty_decisions_source_file
  ON penalty_decisions (source_file);
```

3. Loading JSON into Postgres. File: `verdict/loader/load_to_postgres.py`. Reads each saved JSON file and inserts it, using `ON CONFLICT (source_file) DO NOTHING` — the same safe-reload pattern already used by Gridline's existing Airflow pipelines (e.g. `airflow/dags/driver_standings.py`):

```
def load_one(cur, json_path):
    with open(json_path) as f:
        data = json.load(f)

    filename = os.path.basename(json_path)
    source_file = filename.replace(".json", ".pdf")

    cur.execute("""
        INSERT INTO penalty_decisions
            (source_file, driver_text, own_car, other_car, competitor,
             session, incident_time, turn, fact, infringement, decision,
             penalty_points, reason)
        VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
        ON CONFLICT (source_file) DO NOTHING
    """, (
        source_file, data.get("Driver"), data.get("own_car"),
        data.get("other_car"), data.get("Competitor"), data.get("Session"),
        data.get("Time"), data.get("turn"), data.get("Fact"),
        data.get("Infringement"), data.get("Decision"),
        data.get("penalty_points"), data.get("Reason"),
    ))
```

A Couple of Terms, Explained Simply

PDF parsing — teaching a program to read a document and pick out specific pieces of information, instead of a person reading it by hand.

Regex (regular expression) — a mini pattern language for finding text shapes, like “a number right after the word Turn”, without having to write custom logic for every possible sentence.

JSON — a simple way to save information as labeled pairs (like a name tag: “this label, this value”) in a plain text file.

Database / SQL migration — permanent, organized storage for information (database), created by running a script (a “migration”) that defines its structure once, ahead of time.

That's it — two parts down, five to go. Next up is the hardest one: matching each penalty to the actual speed and position data from that moment in the race.